

深入理解 JavaScript——立即执行函数 (IIFE)



山头人汉波

2022-10-12 上海

📖 阅读 5 分钟

一句话解释

1. 立即执行函数是什么？

立即执行函数就是声明一个匿名函数，并马上调用这个匿名函数

2. 立即执行函数有什么用途

创建一个独立的作用域，这个作用域里面的变量，外面访问不到（即避免"变量污染"）

我们先问自己一个问题：立即执行函数是闭包吗？如果你不能马上回答这个问题，那么不妨往下看

什么是立即执行函数

来自 [MDN](#) 的回答是

IIFE（立即调用函数表达式）是一个在定义时就会立即执行的 JavaScript 函数）。

这样就能形成一个 **块级作用域** 效果

```
(function () {  
    // 块级作用域  
})();
```

这在没有块级作用域的 ES3 时代，是相当普遍的做法

以前有个有名的面试题，如下所示：

```
for (var i = 0; i < 5; i++) {  
    setTimeout(function () {  
        console.log(i);  
    }, 1000 * i);  
}
```

结果是什么, 5, 5, 5, 5, 5, 而且是在每 1 秒打印一个 5

问, 有什么方法让它的结果是 1, 2, 3, 4, 5

我们分析一下为什么会在一开始的时候打印 5, 这是因为 setTimeout 是异步, 要塞进异步队列中, 所以一开始先循环, 循环完了再执行 setTimeout(func, wait)。

所以执行顺序是

```
for (var i = 0; i < 5; i++) {  
    // 赋值 setTimeout(function() { console.log(i) }, 1000 * i)  
    // i 1,2,3,4,5  
}  
// setTimeout 延迟执行, var i 被统一赋值为 5  
setTimeout(function () {  
    console.log(5);  
}, 1000 * 1);  
setTimeout(function () {  
    console.log(5);  
}, 1000 * 2);  
setTimeout(function () {  
    console.log(5);  
}, 1000 * 3);  
setTimeout(function () {  
    console.log(5);  
}, 1000 * 4);  
setTimeout(function () {  
    console.log(5);  
}, 1000 * 5);
```

又因为, for() {} 不会形成块级作用域, 所以会拿最后的值也就是 5 来给每一个 func 中的 console.log(i) 赋值, 最后导致了这样的打印结果

分析完后, 我们要思考一下, 怎么保住 setTimeout 中的变量 i, 通常的办法是通过作用域来保护, 例如用块级作用域来保护 i, 方法是用 let 代替 var。

```
for (let i = 0; i < 5; i++) {  
    // 将 var 改成 let 即可  
    setTimeout(function () {  
        console.log(i);  
    }, 1000 * i);  
}
```

或者用函数作用域来保护，因为函数作用域内的变量，函数外不能访问

```
for (var i = 0; i < 5; i++) {  
    (function (j) {  
        setTimeout(function () {  
            console.log(j);  
        }, 1000 * j);  
    })(i);  
}
```

用 let 的方法的伪代码类似于立即执行函数。代码理解为：

每传入一个变量 i，并立即执行 setTimeout，执行完毕一次后，for 循环中的 i 变为 1，再执行 setTimeout，这样就达到了效果

其原面试题为什么会出现这种结果，本质是 JavaScript 中的 for 循环不能保护 i 被改变，即 for 循环不能形成块级作用域。

通过这题我们能清晰的认知到立即执行函数的用处：**定义时就会立即执行的函数**

IIFE 的延展形态

我们常见的 IIFE 是这样的：

```
;(function() {  
    ...  
})();
```

但是不乏看到这样的代码

```
(function (window) {  
    console.log(window);  
})
```

```
})(window);
```

刚开始我们会很懵逼，这是可以传值？

先看看普通函数怎么运行的

```
var foo = function (name) {  
    console.log(name);  
};  
foo('johan');
```

我们可以在任何地方调用 foo 函数，

之所以要创造“IIFE”，是因为它们是立即调用的函数表达式，这意味着它们会在运行时立即被调用，且我们不会再去调用它，它只运行一次，如下所示：

```
var foo = (function (name) {  
    console.log(name);  
})('johan');
```

甚至，我们可以不用赋值给 foo，因为我们并不会使用 foo

```
(function (name) {  
    console.log(name);  
})('johan');
```

以上例子很好理解吧，自己定义一个匿名函数并且自己传入参数调用

为什么要有 IIFE

原因很简单，为了让一块代码执行且不被其他库影响。在 ES6 的模块出现之前，我们写 JavaScript 不是在 HTML 的 script 标签中书写，就是在 javascript 文件中写代码再通过 script 标签引入。当你写的（全局）方法和别人（第三方库也好，同事的代码也好）相同时，就会有方法覆盖的 bug 存在

所以用 IIFE，保证了每一个 IIFE 中的代码变量不会在全局作用域下被访问，也就起到了变量保护的作用

适用场景

UMD 打包

```
(function (root, factory) {  
    if (typeof define === 'function' && define.amd) {  
        define(factory);  
    } else if (typeof exports === 'object') {  
        module.exports = factory;  
    } else {  
        root.MYMODULE = factory();  
    }  
})(this, function () {  
    //...  
});
```

本质就是把 AMD 和 CommonJS 结合在一起

源码中的立即执行函数

jQuery 中的:

```
(function(window, undefined) {  
    ...  
})(window);
```

underscore 中的:

```
(function(global, factory) {  
    ...  
})(this, (function () {  
    ...  
}));
```

这些大佬库中都用到了立即执行函数来保护库中变量

立即执行函数是闭包吗

回到一开始的问题, IIFE 是闭包吗?

肯定不是，IIFE 是立即执行函数，执行完就被垃圾回收了，怎么会是闭包呢？

什么是闭包？闭包是要利用作用域机制把控私有变量

两者为什么会被人搞混？

因为 IIFE 能起到隔离变量的作用，为模块化没出来前而做的 hack 变量保护机制。而闭包恰恰也能起到隔离变量的效果。所以这两者会被人搞混

那有立即执行函数实现闭包的场景吗？

```
var Module = (function () {  
    var private = '私有变量';  
    var foo = function () {  
        console.log(private);  
    };  
    return {  
        foo: foo,  
    };  
})();
```

```
Module.foo(); // 私有变量  
Module.private; // undefined
```

立即执行函数不是闭包，但是它可以做出闭包效果

三题见真章

第一题

```
(function () {  
    if (typeof name === 'undefined') {  
        console.log('Goodbye ' + name);  
    } else {  
        var name = 'Jack';  
        console.log('Hello ' + name);  
    }  
})();
```

<details>